Khulna University of Engineering Technology

Khulna 9203, Bangladesh

CSE 3200:Electronics Project Design and Development

Project Report

(Smart Vaccum Cleaner Robot)

Presented By:	Supervised By:
Name:Muhammud Tanmin Raiyan Niom Roll: 2009008 Group: A Department of ECE	Name:Dr.Md Foysal Professor,Dept of ECE KUET,Khulna
Name:Mehrab islam Mon Roll: 2009024 Group: A Department of ECE	

Date of Submission: 21-1-2025

Title: Arduino-based Smart Vacuum Cleaner Robot.

Abstract:

In the present era, we are all so preoccupied with our jobs that we lack the time to thoroughly clean our homes. Simply purchasing a domestic vacuum cleaner robot that will clean our home with the push of a button will solve the issue. But the cost is a problem that all such commercial goods share. We, therefore, made the decision to construct a basic floor cleaner robot, which is not only easy to construct but also quite inexpensive when compared to commercially produced items on the market. With the help of a few components, we can make this smart vacuum cleaner for ourselves as well. Additionally, we used an ultrasonic sensor to enable the robot to avoid obstacles and move freely until the room is thoroughly cleaned. Our goal with this project was to make a vacuum cleaner robot based on Arduino which is less costly and easy to use and thankfully, we have achieved our goal.

Introduction:

Arduino UNO is a microcontroller board based on the ATmega328P. It has 14 digital input/output pins of which 6 can be used as PWM outputs, 6 analog inputs, a 16 MHz ceramic resonator, a USB connection, a power jack, an ICSP header and a reset button. The open-source Arduino platform is used to build interactive electronics projects. An integrated development environment or IDE is a piece of software that runs on our computer and is used to write and upload computer code to the Arduino microcontroller board. Additionally, the Arduino Uno does not require a hardware component to upload fresh code to the board. A USB cord and the Arduino IDE which utilizes a simpler version of C++ to write code, we can quickly load a code onto the board.

An autonomous robotic vacuum cleaner, sometimes known as a robovac or roomba as a generic trademark, features a restricted vacuum floor cleaning system coupled with sensors, robotic drives, and programmable controllers and cleaning routines. Even the lightest container vacuums weigh substantially more than robotic vacuums, which are often smaller and lighter. A robotic vacuum cleaners' drawback is that, because of their size, it takes a long time to clean an area. As most of the vacuum cleaners in the market are too expensive to buy so, we, therefore, made the decision to construct a basic floor cleaner robot using Arduino Uno, which is not only easy to construct but also quite inexpensive when compared to commercially produced items on the market. This vacuum cleaner robot can be made at home with a few components which are not too expensive at all and available in the market. In this project report, a brief discussion on the methods of making an Arduino-based smart vacuum cleaner robot is done.

Literature Review:

1. Embedded Systems and Electronics are used to create a four wheeled vacuum cleaner that can automatically avoid obstacles while cleaning the floor at the same time and help us in keeping our home or workplace neat and tidy. [1]
2. The Arduino Vacuum Cleaner will be portable and useful, including ultrasonic sensors and an IR proximity sensor to assist it avoid obstacles and falling downstairs. These are the most crucial details. Additionally, it will have IR proximity sensors to help it avoid falling downstairs and ultrasonic sensors to help it avoid obstacles. For those who lack the time to thoroughly clean their homes, this robot is a perfect option. [2]
3. This vacuum cleaner works on a pre-defined code inserted in Arduino UNO. Whenever an RC car encounters any obstacle, it turns to the side where the distance between obstacle and car is more. This project helps collect the dust using a vacuum cleaner made using CPU fan and batteries without human intervention thereby reducing the hazards to human health. This is a simple and cost-effective cleaner. However, using a detachable bag may be better as removing the dust becomes simpler. [3]
4. The development of a robot vacuum cleaner utilizing an Arduino and an android smartphone is covered in this paper. The key concept is to have a sensor that can detect items and output data to an Arduino, which will control the movement of the robot vacuum. Targeted functions require interaction and excellent sensors. Automatic vacuum cleaning mode and manually operated cleaning mode are the two newly designed functions. A robot prototype has been created and put to the test in a lab. [4]

Components:

1. Arduino Uno
2. Arduino Motor Shield
3. Gear Motors
4. DC Motor
5. Wheels
6. LED Display
7. Drone Motor for Vaccum
8. Plastic Fan
9. Servo Motor
10. Ultrasonic Sensor
11. IR Sensor
12. Switch
13. Battery
14. Plastic Bottle
15. Plastic Pipe

Working Principles:

The smart vacuum cleaner robot uses a microcontroller which is used for interfacing hardware and software. To do the same, a USB cable is required. Once the board is embedded with the code, it can be operated by a battery supply without using any PC or laptop. Two 3.7V batteries are used to run the device. Gear motors and a DC motor are used to make the machine move along using the wheels. These motors essentially are the key components in this vacuum cleaner. Arduino motor shield is used to run different types of motors. The direction and speed of motors depends on the motor shield, as the shield is embedded on Arduino UNO board and the speed and direction can be controlled by coding in Arduino IDE. The ultrasonic sensor, HC-SR04 is used for measuring distance and finding obstacles. It relates to a servo motor and uses its rotation to do the job. Plastic bottles and pipes are used to collect the dirt. Jumper wires are used where necessary to complete the whole circuitry.

Hardware Layout:

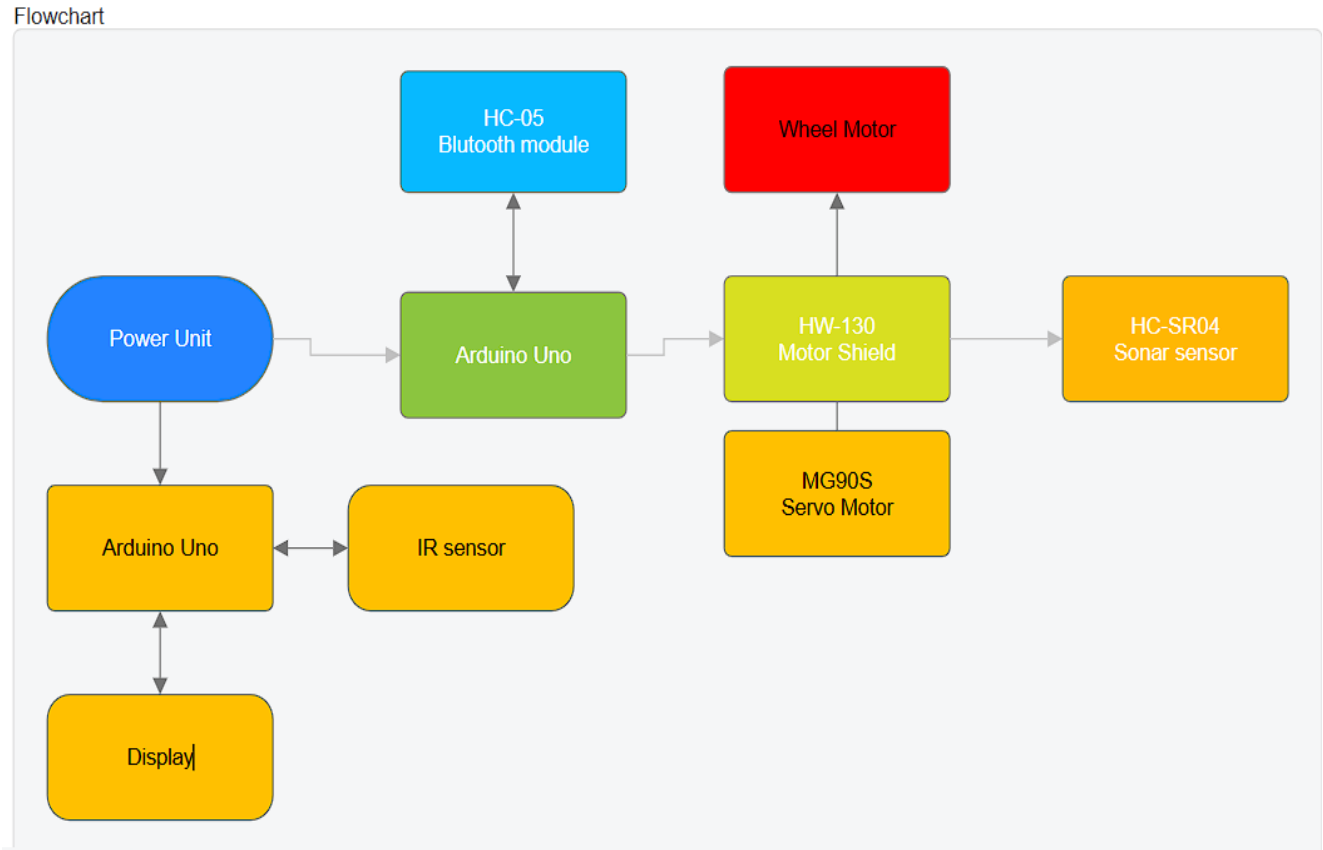

Figure 1: Hardware layout for smart vacuum cleaner robot.

Arduino IDE Code:

File Edit Sketch Tools Arduino Uno sketch_jan20i | Arduino IDE 2.3.4

sketch_jan20i.ino

```
#include <AFMotor.h>
#include <NewPing.h>
#include <Servo.h>

#define TRIG_PIN A0
#define ECHO_PIN A1
#define MAX_DISTANCE 200
#define MAX_SPEED 150 // sets speed of DC motors
#define MAX_SPEED_OFFSET 20
#define DETECTION_THRESHOLD 30 // Reduce detection threshold for earlier obstacle detection

NewPing sonar(TRIG_PIN, ECHO_PIN, MAX_DISTANCE);

AF_DCMotor motor1(1, MOTOR12_1KHZ);
AF_DCMotor motor2(2, MOTOR12_1KHZ);
Servo myservo;

boolean goesForward = false;
boolean systemEnabled = true; // Bluetooth-controlled system state
boolean isManualMode = false; // Mode state: false = automatic, true = manual
int distance = 100;
int speedSet = 0;

void setup() {
  myservo.attach(10);
  myservo.write(90); // Center the servo initially
  delay(2000);
  distance = readPing();
  delay(100);

  Serial.begin(9600); // Initialize hardware serial for HC-05 communication
}
```

File Edit Sketch Tools Arduino Uno sketch_ja

sketch_jan20i.ino

```
        break;
    }
  }

  void automaticObstacleAvoidance() {
    int distanceR = 0;
    int distanceL = 0;
    delay(40);

    distance = readPing(); // Read sensor before making a decision

    if (distance <= DETECTION_THRESHOLD) { // Obstacle detected
      moveStop();
      delay(100);
      moveBackward();
      delay(300);
      moveStop();
      delay(200);

      distanceR = lookRight();
      delay(200);
      distanceL = lookLeft();
      delay(200);

      if (distanceR > DETECTION_THRESHOLD || distanceL > DETECTION_THRESHOLD) {
        if (distanceR > distanceL) {
          turnSlightRight();
        } else {
          turnSlightLeft();
        }
      } else {
        // If neither side is free, move backward and reevaluate
```

File Edit Sketch Tools Arduino Uno

sketch_jan20i.ino

```
        // If neither side is free, move backward and reevaluat
        moveBackward();
        delay(500);
      }
    } else {
      moveForward();
    }
  }

  bool checkForObstacle() {
    distance = readPing();
    return distance <= DETECTION_THRESHOLD;
  }

  int lookRight() {
    myservo.write(45); // Turn servo to the right
    delay(300); // Reduced stabilization time
    int distance = readPing();
    delay(100);
    myservo.write(90); // Reset servo to center
    return distance;
  }

  int lookLeft() {
    myservo.write(135); // Turn servo to the left
    delay(300); // Reduced stabilization time
    int distance = readPing();
    delay(100);
    myservo.write(90); // Reset servo to center
    return distance;
  }
```

sketch_jan20i.ino

```
        delay(5);
      }
    }
  }

  void moveBackward() {
    goesForward = false;
    motor1.run(BACKWARD);
    motor2.run(BACKWARD);

    int min_speed = 50;  // Increase this value if needed
    for (speedSet = min_speed; speedSet < MAX_SPEED; speedSet += 2) {
      motor1.setSpeed(speedSet);
      motor2.setSpeed(speedSet);
      delay(5);
    }
  }

  void turnSlightRight() {
    motor1.run(FORWARD);
    motor2.run(BACKWARD);
    delay(200); // Reduced turn duration
    moveForward();
  }

  void turnSlightLeft() {
    motor1.run(BACKWARD);
    motor2.run(FORWARD);
    delay(200); // Reduced turn duration
    moveForward();
  }

```

File Edit Sketch Tools Arduino Uno sketch

sketch_jan20i.ino

```
void handleManualControl(char command) {
  switch (command) {
    case 'F': // Up Arrow - Forward
      if (checkForObstacle()) {
        Serial.println("Obstacle Ahead! Stopping...");
        moveStop();
      } else {
        moveForward();
      }
      break;
    case 'B': // Down Arrow - Backward
      moveBackward(); // Allow backward movement without obstacle checking
      break;
    case 'L': // Left Arrow - Turn Left
      if (checkForObstacle()) {
        Serial.println("Obstacle Detected on Turn! Stopping...");
        moveStop();
      } else {
        turnSlightLeft();
      }
      break;
    case 'R': // Right Arrow - Turn Right
      if (checkForObstacle()) {
        Serial.println("Obstacle Detected on Turn! Stopping...");
        moveStop();
      } else {
        turnSlightRight();
      }
      break;
    case 'S': // Stop
      moveStop();
      break;
```

Hardware Implementation:

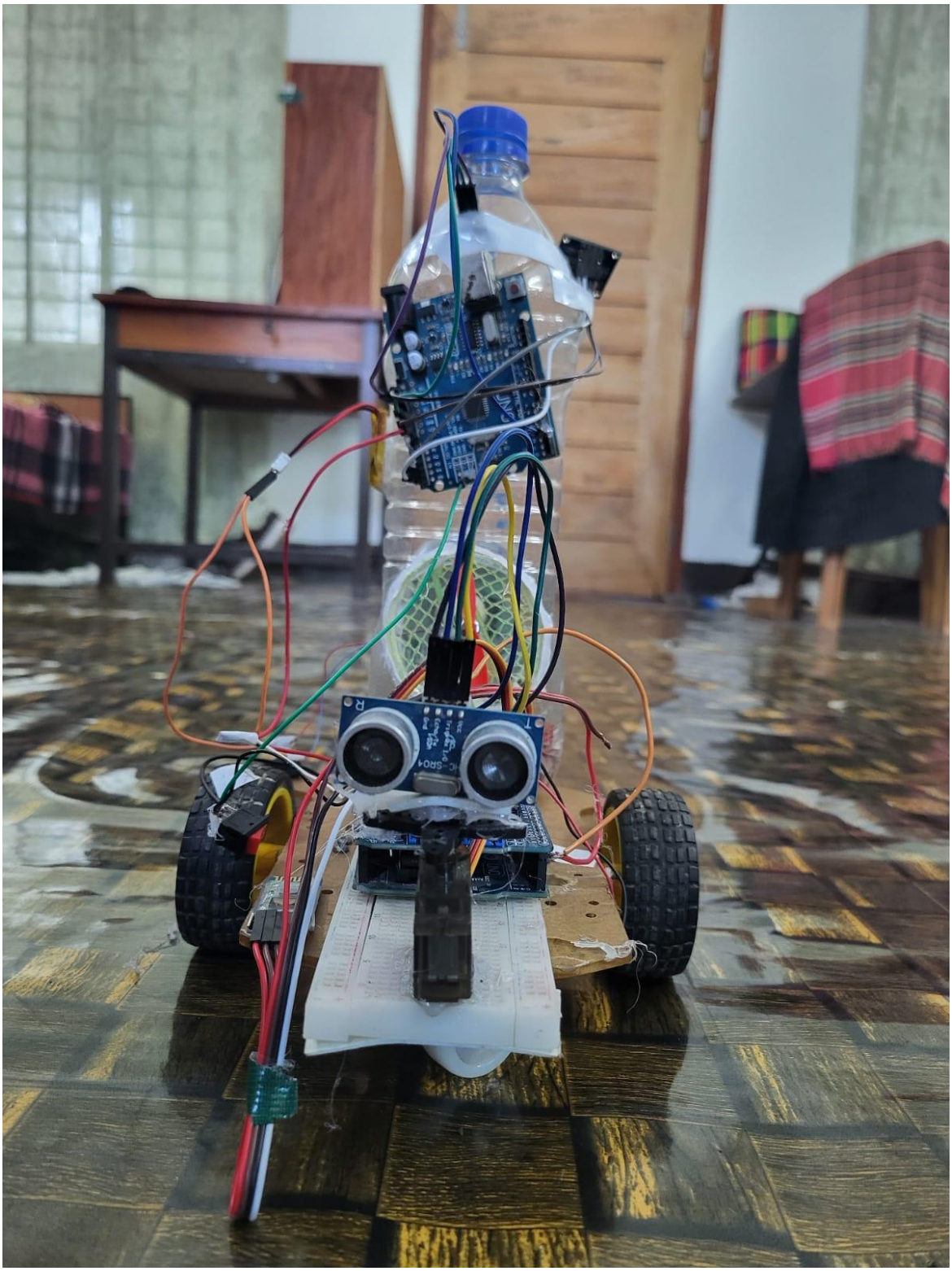

Figure 2: Implemented Arduino-based smart vacuum cleaner robot.

Limitations:

There can be several limitations of an Arduino-based vacuum cleaner robot.

1. Battery life: The battery life of the robot depends on the capacity of the batteries used and the power consumption of the motors and other components. If the batteries run out of charge, the robot will stop working and need to be recharged.
2. Limited suction power: The suction power of the vacuum cleaner is limited by the size of the plastic fan used and the power of the 6V DC motor. This affects the robot's ability to clean larger debris or deep clean carpets.
3. Obstacle detection limitations: The ultrasonic sensor used for obstacle detection has a limited range and may not detect smaller obstacles or obstacles that are placed at an angle to the sensor. This could result in collisions or missed spots during cleaning.
4. Navigation limitations: The robot's ability to navigate and cover all areas of a room may be limited by its programming and the accuracy of the sensor readings. The robot may miss spots or get stuck in tight spaces.
5. Maintenance: The robot may require regular maintenance to ensure that it is functioning properly. This could include cleaning the fan, replacing worn-out wheels or motors, and fixing any wiring or programming issues that may arise.

Cost Analysis:

Sl.	Component	Quantity	Price (Taka)
1	Arduino Uno	2	1950
2	Arduino Motor Shield	1	290
3	Servo Motor	1	230
4	Ultrasonic Sensor	1	140
5	Gear Motors	2	180
6	DC Motor	1	80
7	Battery	3	270
8	Wheels	2	130
9	Cardboard	1	150
10	Plastic Fan	1	10
11	Switch	2	20
12	Plastic Pipe	3	20
13	Drone Motor for Vaccum	1	70
14	Display	1	110
15	Black Tape	1	20
16	IR Sensor	1	70
		Total Cost	**3740**

The components did not cost much, and they were easily available in the market.

Conclusion:

In our modern life, we don't get much time to do our household work. We always try to find a way to save our time and let technology do the work for us. Cleaning our home is one of those. It is increasingly difficult to clean homes and the surrounding area with the present hectic routine. There are vacuum cleaners on the market right now, but they need human intervention. Therefore, it is imperative to implement a vacuum cleaner that operates automatically. Keeping this in mind, we have tried to construct an Arduino-based smart vacuum cleaner robot to clean the dirt and save our valuable time as well. There are other smart vacuum cleaners available in the market but an Arduino-based vacuum cleaner like ours does costs a lot less than the other ones, the components used to build the vacuum cleaner are available everywhere and it is also quite simple to make. Though an Arduino-based smart vacuum cleaner robot has some limitations, it also offers some useful features, and it does not need human intervention. Someone just needs to turn on the switch once and the smart vacuum cleaner robot will do the rest of the job on its own. This project uses an Arduino Uno and batteries to create a vacuum cleaner that collects dust automatically, lowering the risks to human health. This cleaner is easy to use and reasonably priced. To sum up, we can say the goal of our project was achieved.

Future Endeavors:

There are several potential future endeavors for this Arduino-based vacuum cleaner robot project:

1. Improving suction power: One way to improve the robot's cleaning ability is to increase the suction power. This could be done by using a more powerful DC motor or a larger plastic fan. Another option could be to add a HEPA filter to the robot to improve air quality and capture more fine particles.
2. Adding smart features: The robot could be equipped with additional sensors, such as cameras or lidar, to improve navigation and obstacle avoidance. This could also enable the robot to map out a room and remember areas that need to be cleaned.
3. Improving battery life: The robot could be equipped with a more powerful battery or a wireless charging system to extend its battery life and reduce the need for manual recharging.

Overall, there are many opportunities to expand and improve upon this Arduino-based vacuum cleaner robot project, and it has the potential to be a valuable tool for home cleaning and automation.

References:

[1] DIY smart vacuum cleaning robot using Arduino (no date) Arduino Based Simple Vacuum Cleaning Robot. Available at: https://circuitdigest.com/microcontroller-projects/diy-arduino-based-smart-vacuum-cleaning-robot .

[2] Build your own arduino based smart vacuum cleaner robot for automatic floor cleaning (no date) Circuit Digest - Electronics Engineering News, Latest Products, Articles and Projects. Available at: https://circuitdigest.com/microcontroller-projects/build-your-own-arduino-based-smart-vacuum-cleaning-robot-for-automatic-floor-cleaning

[3] Author links open overlay panelManasa M and schedule, A.I.the current hectic (2021) Smart vacuum cleaner, Global Transitions Proceedings. Elsevier. Available at: https://www.sciencedirect.com/science/article/pii/S2666285X21000790

[4] Design and implementation of vacuum cleaner robot using Arduino and ... (no date). Available at: https://www.researchgate.net/publication/327682474_Design_and_implementation_of_vacuum_cleaner_robot_using_Arduino_and_smartphone